

Support Triage Assistant — Collected Learnings

A running log of what was built and, more importantly, what was learned while building a support-ticket triage system with the Anthropic API — and then folding it into the ninjamountain monorepo and putting it under a real evaluation harness.

The original day-by-day notes (Days 1–2) live in [experiments/LESSONS.md](#); this document collects them with everything that came after.

1. The project at a glance

The triage pipeline takes a support ticket (**subject** + **body**) and returns a structured judgment from Claude:

- **category** — one of bug, config, billing, how_to, feature_request
- **severity** — one of low, medium, high, critical
- **summary** — one sentence
- **suggested_first_response** — a draft reply
- **needs_engineering_escalation** — boolean

It never throws: every call resolves to a typed **TriageOutcome** (success with a result, or failure with a reason). It retries transient API errors with backoff, and re-prompts the model with a corrective hint when output fails to parse or validate.

Architecture (single responsibility per file):

- **client.ts** — the Anthropic API call (typed errors, exponential backoff)
 - **parse.ts** — extract JSON from raw model output
 - **validate.ts** — validate output against the contract (returns errors, never throws)
 - **triage.ts** — orchestration: the retry + correction loop
 - **types.ts** — domain and return types
-

2. Days 1–2 — Foundation (summary)

Day 1 (JavaScript). Built the core pipeline with a two-layer retry strategy: API failures retried in **client.js** with backoff; output failures retried in **triage.js** with a corrective hint appended to the next prompt. The validator returns **{ valid, errors }** instead of throwing, keeping the orchestration layer in control.

Key bug caught in review: a typo in the system prompt (**suggested_fist_response**) meant every call failed validation on the first attempt and burned a retry — and the type system could not catch it because the field name lived inside a string literal.

Day 2 (TypeScript port). Same logic, but types replaced documentation: a discriminated union (`TriageOutcome`) the compiler enforces at every call site, `unknown` for unvalidated model output, and typed SDK errors (`instanceof APIError`) instead of status guessing. The lasting lesson: **static types stop at the string boundary**. The prompt and the `TriageResult` interface can drift without the compiler noticing — which is exactly why runtime validation stays essential.

3. Day 3 — Labeling the gold dataset

Day 3 was supposed to be one task: make sure the labels accurately reflect the tickets. It turned out to be the foundation for everything that follows.

The labels are the asset, not the code. A scoring script is easy; a trustworthy set of gold labels is the hard, valuable part. The initial `labels.json` was placeholder data (18 of 19 entries identical), which would have produced meaningless — even deceptively good — scores. Garbage labels in, garbage metrics out.

Lessons:

- Read the full ticket body before labeling — the subject is not enough.
 - Severity is far more subjective than category; calibrate it deliberately (a billing question is low; an account suspension is critical).
 - Hand-labeling 20 tickets surfaces the genuinely ambiguous cases — and those ambiguities are signal, not noise. They tell you where your definitions are underspecified.
-

4. Day 4 — Evaluation-driven iteration

This was the heart of the project: building a scored accuracy report and using it to drive changes. The metric journey:

Stage	Category	Severity exact	Severity MAE	Trustworthy?
First baseline (temp 1)	90%	55%	0.50	No — noisy
Severity rubric v1 (temp 1)	90%	50%	0.55	No — noisy
Rubric v2 + temperature 0	85%	45%	0.60	Yes — stable
Reconciled labels to one rule	90%	85%	0.15	Yes
"No workaround = high" policy	85%	90%	0.10	Yes
Category rubric + T18 relabel	90%	85%	0.15	Yes

The numbers going **down** at the temperature-0 step was the most important moment: it revealed that the earlier higher scores were partly luck, not skill.

The lessons, in order of how hard they hit:

- 1 **Vibes do not scale — measure.** "The triage feels good" becomes "category is 90%, severity is stuck at 50%, and here is exactly where it fails."
- 2 **Pin temperature: 0 for evaluation.** At the default temperature the model rolls dice; run-to-run variance on 20 tickets is several points. You cannot measure a prompt change while the ground moves under you. A lower-but-stable score beats a higher-but-random one.
- 3 **A failing eval often means the labels are wrong, not the model.** Two tickets (T02 a suspected-fraud billing case, T18 a platform ignoring a correct setting) were gold-label errors the score exposed. The model was right; the labels were not.
- 4 **Make the specification self-consistent.** Most of the severity jump (45% to 85%) came from reconciling the labels to one written rule — not from the model improving. Being honest about **why** a number moved matters as much as the number.
- 5 **A shared prompt couples tasks.** Editing the severity wording shifted category predictions; adding a category rubric nudged a severity label. Always score the **whole** report after any change, not just the metric you were targeting.
- 6 **Written policy beats per-case instinct.** A severity rubric is operational policy. Its value is not that it is right on every ticket — it is that it is **decided**, which buys consistency, speed, and defensibility. "No workaround = high" is a clean, teachable line that ends the per-ticket debate.
- 7 **There is a small-data ceiling.** Past a point, every prompt edit just trades one borderline ticket for another, and the metric swings because the sample is tiny. The next lever is a bigger dataset, not more prompt clauses.

Where it landed: category 90% (config recall 1.0), severity 90% exact / 100% off-by-one / MAE 0.10. The remaining misses are genuine model weaknesses (bug reports phrased as polite questions get read as `how_to`) — the honest, interesting kind of leftover.

5. Engineering & workflow lessons

Folding the standalone experiment into ninjamountain taught a parallel set of lessons that had nothing to do with the model:

- **One source of truth.** The triage logic became a workspace package (`@ninjamountain/triage`) consumed by the web app via `transpilePackages`, so there is no duplicated copy to drift. The Python/JS ports were kept as frozen reference snapshots under `experiments/`, explicitly not wired into the build.
- **Preserve before you retire.** The old standalone repo's only copy of the TS+Python work was an unpushed commit. Push first, then archive, then delete — never destroy the only copy.
- **.env is not auto-trusted.** A subtle one: `dotenv` will not overwrite an already-defined environment variable, so an empty `ANTHROPIC_API_KEY` in the environment silently beat the value in `.env`. Loading with `override: true` (and an explicit path) fixed it. Also: Next.js `.env.local` changes require a dev-server restart.
- **Keep secrets out of git.** API keys live only in gitignored `.env` / `.env.local` files; a pre-publish scan confirmed nothing leaked before the repo went public.

- **A real test loop is worth building.** Two harnesses — the web UI (`/projects/triage`) and a CLI (`npm run triage`) — make iterating far faster than a hardcoded script.
- **Errors in files you didn't touch point to the environment.** While adding the OpenAI provider, `npm run typecheck:web` failed with `Cannot find name 'process'` (TS2580) and `Cannot find module 'node:url'` (TS2307), including in files the change never touched. The cause was the local install, not the code: `package-lock.json` pins a separate `@types/node` under `apps/web/node_modules/`, and that folder was missing on disk. The lockfile was correct, so CI would have passed. `npm ci` from the repo root rebuilt `node_modules` to match the lockfile exactly (the same command CI runs) and fixed it. Rule of thumb: when type errors show up in unchanged files, check the install before the code, and use `npm ci` as the "make my machine match CI" reset.

6. What is this field called?

The evaluation work has a name — several, depending on how academic you want to be.

The umbrella term is model evaluation, or simply "evals." Applied to large language models it is often called **LLM evaluation**, and the practice of building a metric and letting it drive changes is **evaluation-driven development** (the ML cousin of test-driven development).

It is not one field but a convergence of several:

- **Statistical classification / pattern recognition** — the source of accuracy, precision, recall, F1, and the confusion matrix. Our category scoring is textbook multi-class classification evaluation.
- **Information retrieval (IR)** — where precision and recall originated (how many relevant documents did a search return, and how many of the returned ones were relevant).
- **Psychometrics & measurement theory** — the science of measuring fuzzy human judgments reliably. This is where label quality, *inter-annotator agreement*, and *reliability vs validity* come from. Our whole Day-3/4 struggle — getting labels consistent enough to trust — is a psychometrics problem.
- **Statistics & experimental design** — sampling, variance, significance, and A/B testing. The "is this change real or noise?" question is a statistics question, and the small-data ceiling is about sample size.
- **Ordinal data analysis** — severity is an *ordinal* variable (low < medium < high < critical), so an off-by-one miss is not as bad as off-by-three. That is why we tracked mean absolute error, not just exact match.

For this specific project, the precise label is **supervised text-classification evaluation against a gold-labeled dataset** — with one ordinal target (severity) and one nominal target (category).

7. Glossary

Term	Meaning
Ground truth / gold label	The human-assigned correct answer a prediction is scored against.
Golden dataset	The curated set of inputs plus gold labels used for evaluation.
Accuracy	Fraction of predictions that exactly match the gold label.

Term	Meaning
Precision	Of the items predicted to be class X, how many truly were X.
Recall	Of the items that truly are class X, how many were caught.
F1	Harmonic mean of precision and recall — one number balancing both.
Support	How many gold examples exist for a class (the denominator behind recall).
Confusion matrix	A grid of "predicted X but actually Y," exposing systematic mistakes.
Nominal variable	A category with no order (bug vs billing).
Ordinal variable	A category with a meaningful order (low < ... < critical).
Mean absolute error (MAE)	Average distance between predicted and true rank — for ordinal targets.
Off-by-one	An ordinal prediction one step from the truth (high vs critical).
Inter-annotator agreement	How consistently independent humans assign the same labels; a measure of label reliability.
Temperature	Sampling randomness; 0 makes the model (near-)deterministic.
Baseline	The reference score a change is measured against.
Regression	A change that makes a previously-good metric worse.
Label noise	Errors or inconsistencies in the gold labels themselves.
Class imbalance	Some classes having far more examples than others, skewing metrics.
Eval harness	The script/tooling that runs predictions and computes the report.
Evaluation-driven development	Building a metric first, then iterating against it.

8. Command reference

Command	What it does
<code>npm --workspace @ninjamountain/triage run triage</code>	Triage a built-in sample ticket.
<code>npm ... run triage -- --id T02</code>	Triage one dataset ticket by id.
<code>npm ... run triage -- --all</code>	Triage every ticket in the dataset.
<code>npm ... run score</code>	Full scored accuracy report vs gold labels.
<code>npm ... run typecheck</code>	Type-check the package.
<code>npm run dev:web</code>	Start the Next.js app; UI at /projects/triage.

9. Day 7 — Knowledge base assembly (v2 / RAG, in progress)

Picking the 14-day plan back up at Day 7 after a long gap (Days 1–6 covered above). Goal: 10–20 KB articles to ground retrieval for `suggested_first_response`.

Embedding model: Voyage `voyage-3` — Anthropic's recommended embeddings partner, keeps a clean "Claude classifies, Voyage retrieves" story. Got an API key, confirmed it works with a real call (1024-dimension embedding returned for a test string) before writing any KB content.

Grounding, not invention. Rather than writing 15 plausible-sounding articles from scratch, drafted them directly from `experiments/data/grounding/*.md` — real Twilio product specifics (error codes, terms, failure modes) already assembled for synthetic ticket generation (`DATASET_GENERATION.md` §4) and already matching the dataset's actual ticket categories (Flex, TaskRouter, Voice/TwiML, Messaging, Verify, Lookup, 10DLC, Auth, Account/billing, SendGrid).

The review pass caught three real factual errors, not just style — see `[KB_REVIEW_NOTES.md](./KB_REVIEW_NOTES.md)` for the full reasoning behind every change:

- 1 TaskRouter's task-cancellation threshold — drafted as 10 rejections, actually 1,000 per current docs (10 is explicitly flagged outdated).
- 2 Verify's Silent Network Auth codes 60534 vs. 60540 need *different* fallback handling, not identical treatment — drafted them the same.
- 3 Studio's 81026 (widget limit) — drafted as a publish-time warning; it's actually an execution failure blocking every run until fixed.

The lesson, stated plainly: AI-drafted domain content is a fast first draft, not a trustworthy final answer — even when it's grounded in real reference material, it can still assert something confidently wrong. The review-and-verify step is what actually makes the KB usable, the same discipline as Day 3's "label blind" rule: don't trust a number (or a claim) until it's been checked against ground truth.

10. Days 8–9 — Embedding, retrieval, and a real RAG eval (v2 complete)

Day 8 — wiring it up. `scripts/embed.ts` embeds every `kb/*.md` body via Voyage and writes `kb/embeddings.json` (checked into git like a lockfile — regenerated manually via `npm run kb:embed` when KB content changes, rather than calling Voyage on every deploy). `src/retrieve.ts` embeds the incoming ticket and does cosine-similarity search against those embeddings, returning the top-3 matches. `triage.ts` retrieves once per ticket (reused across output-retries, since a retry means the model's *output* was malformed, not that the ticket or its relevant KB context changed), prepends the snippets to the prompt, and the model returns `kb_citations: string[]` — only the article ids it actually drew on, not everything it was shown.

A genuine retrieval-ranking nuance, caught by hand-checking a real query before wiring anything further: querying "inbound SMS webhook stopped firing" scored `kb-006` (Webhook/TwiML errors — very webhook-dense text) *above* `kb-004` (SMS delivery, which only mentions "inbound webhook not firing" in one paragraph), even though `kb-004` is arguably the more topically correct article for an SMS-specific issue.

Cosine similarity scores the *whole* article's semantic content — a short, topically-focused article can outscore a longer, more topically-diverse one that only touches the exact match briefly. Not a bug: both landed in the top-3, so the model still saw both and picked correctly (confirmed later in the real pipeline run — see below).

A real bug, not just a design nuance: `scripts/rag-eval.ts` (below) hit Voyage's free-tier rate limit (3 requests/minute without a payment method on file) and crashed outright on the first run. Fixed by factoring the Voyage call into a shared `src/embeddings-client.ts` used by both `embed.ts` and `retrieve.ts`, with retry-on-429 exponential backoff — the same category of fix `client.ts` already has for Anthropic's API, just tuned for a much stricter limit (20s/40s/60s backoff instead of 1s/2s/4s). Confirmed working for real on the actual eval run, not just in theory: it hit 429 twice, backed off, and completed successfully instead of crashing.

Day 9 — does retrieval actually help? Ran `scripts/rag-eval.ts` — the same 5 tickets, with and without retrieval, read side by side (a qualitative comparison, not a scored metric; category/ severity don't depend on RAG). Picked a deliberate mix: three tickets expected to have an on-topic KB article, two expected not to.

Ticket	Result	What actually happened
T04 — SSO/Okta	Retrieval prevented a real factual error.	Without retrieval: "the OAuth option... not typically used for agent login" — wrong; Enhanced SSO *is* the OAuth-based flow Twilio recommends for Flex 2.5.x+. With retrieval (kb-009): correctly distinguished Enhanced (OAuth) vs. Legacy (SAML) SSO. Traces directly back to a Day 7 fact-check catching this exact nuance.
T13 — Verify OTP	Retrieval fixed a domain mixup.	Without retrieval: cited SMS/Messaging error codes (30003, 30006, 30034) for a *Verify*-specific issue — wrong domain. With retrieval (kb-010 , kb-004 , kb-012): correct Verify codes (60205, 60207, 60203) plus the 10DLC angle.
T19 — 20003 after key rotation	Neutral.	Both versions comparably solid — a common enough error that the model's own training already covers it well.
T07 — feature request (pause/resume routing)	Helped somewhere unexpected.	Predicted a clean miss (it's a feature request, not troubleshooting) — the model cited kb-008 anyway and used it to suggest a concrete workaround (a custom Worker Activity mapped separately in WFM) the without-retrieval version didn't offer.
T16 — SDK v1→v2 migration	Correctly abstained.	kb_citations: <code>[]</code> — no relevant article exists, and the model didn't force a citation. Comparable quality either way — exactly the desired "miss" behavior, not a failure.

Honest verdict: retrieval meaningfully helped 3/5 (two of them by preventing a real, specific factual error, not just "sounding more informed"), was neutral 2/5, and never made a response worse in this sample. Worth being honest that a larger eval could surface a harmful case this one didn't — five tickets is a qualitative spot-check, not a statistically powered claim.

The lesson underneath all of it: the two cases where retrieval clearly won weren't generic "more context is better" wins — they were cases where the model's **un-grounded** answer was subtly, confidently wrong (dismissing OAuth as irrelevant; citing the wrong product's error codes), and the retrieved article corrected exactly that. That's a more specific, more defensible RAG story than "it made the answers better" — and it's the same discipline as Day 7's KB review: verify against ground truth, don't trust confident-sounding output at face value.

11. Days 10–11 — Tool-use agent (v3 complete)

Day 10 — the shift from pipeline to agent. Everything through v2 (RAG) is a **fixed** sequence: retrieve, then call the model once, then validate. `src/agent.ts` is the first place the sequence itself becomes the model's decision — four tools (`search_kb` real, wrapping Day 8's retrieval; `get_customer_account`, `check_recent_tickets`, `escalate_to_engineering` mocked, since the agent **pattern** is the interview story, not the data source) and a loop: send the ticket, and if the model responds with `stop_reason: "tool_use"`, execute whatever it asked for, feed the results back, and repeat — capped at 6 iterations — until it returns final JSON in the same `TriageResult` shape v1/v2 already use.

First real run, on T01 (the SMS webhook ticket, whose body includes a phone number that matches a mock account): the model called all three of `get_customer_account`, `check_recent_tickets`, and `search_kb` **in parallel** within iteration 1, then produced a correct final answer in iteration 2 — citing the customer's enterprise plan and a recent related ticket (`T-8821`) directly in its suggested reply. That's a materially better answer than v2 could produce, because v2 has no channel for "does this account have relevant history" at all.

Day 11 — proving the exit paths, not just the happy path. Two things that had never actually fired before this:

- 1 The mocked "not found" branch.** Ran T04 (an SSO question with no phone/email/SID anywhere in the body) expecting to see `get_customer_account` return the "no account found" string. Instead the model didn't call it at all — it recognized there was nothing to look up and skipped straight to `search_kb`. Not a bug, but a real observation: **an agent's tool-skipping means some of your code paths are only exercised on demand.** The "not found" branch in `mock-data.ts` is still only proven correct by types and a manual read, not a live run — a gap a fixed pipeline wouldn't have, because a fixed pipeline runs every step every time.
- 2 `max_iterations`.** Temporarily set the cap to 1 and re-ran T01, which we already knew needs 2 iterations for a real answer. It hit the cap cleanly: `{ ok: false, reason: "max_iterations", toolLog: [...] }` — no hang, no crash, and the 3 tool calls already made were still in the log. Reverted the cap to 6 immediately after.

What's actually different about debugging an agent vs. a pipeline. For `trriage.ts`, a failure has a small, known set of causes tied to a specific step — the API call failed, the JSON didn't parse, the schema didn't

validate — and the fix is local to that step. For `agent.ts`, the question is no longer just "did this step fail" but **"was the model's decision at each step reasonable"** — T04 not calling `get_customer_account` wasn't a failure to diagnose, it was correct behavior that happened to look, from the outside, like a step being skipped. That means the `toolLog` isn't just for the demo — it's the primary debugging artifact: without it, a wrong final answer is impossible to attribute to "skipped a tool it needed" vs. "had the right information and drew the wrong conclusion from it." It also means the failure surface itself is bigger: a pipeline can only fail at "API call" or "output validation," but an agent adds a new first-class failure mode — not converging at all — which is why `max_iterations` is a typed `AgentOutcome` reason, not an afterthought.

12. Days 12–13 — MCP server (v4 complete)

Day 12 — a real server, not a bigger mock. `src/mcp-server.ts` re-exposes the three mocked tools from Day 10 (`get_customer_account`, `check_recent_tickets`, `escalate_to_engineering`) as an actual MCP server over stdio, using `@modelcontextprotocol/sdk`'s `McpServer/registerTool` — same mock data (`mock-data.ts`), but now reachable only through the protocol, not a direct function call. Verified with the MCP Inspector before writing a single line of client code: ran `npm run inspect:src/mcp-server.ts`, connected, saw all three tools listed with schemas pulled from the server itself, and called `get_customer_account` directly — confirming the server worked in isolation before trusting it inside the agent loop.

Day 13 — wiring the agent to consume it for real. `src/mcp-client.ts` spawns `mcp-server.ts` as a subprocess per agent run (`StdioClientTransport`), and `agent.ts` now builds its tool list as `[SEARCH_KB_TOOL, ...await mcp.listTools()]` — the three MCP tools are no longer hardcoded anywhere in `agent.ts`; they're discovered from the server's own `tools/list` response every time. `tools.ts` shrank down to just `search_kb` (which stays in-process — it needs the Voyage pipeline's own env vars), closing the "two places have to agree on the schema" gap the Day 10 version had, since now there's exactly one place each tool's schema is defined.

Re-ran the same T01 scenario end-to-end through the real server and got a materially different — and better — result than any earlier run: the model called `escalate_to_engineering` for the first time in the whole project (`ESC-4110`, severity `high`), consistent with `needs_engineering_escalation: true` in the final JSON, and referenced the escalation id directly in its reply. That tool had existed since Day 10 but never actually fired in a live run until it went through MCP — a reminder that a tool being schema-correct and a tool being exercised are two different claims.

What actually changed, concretely, going from in-process to MCP:

- **Discovery replaced duplication.** Day 10's `TOOLS` array in `tools.ts` and the `switch` in `executeTool` had to independently agree with each other. Day 13's `agent.ts` has zero knowledge of what the three MCP tools even look like — it asks the server at connect time. Changing a tool's input schema now means editing `mcp-server.ts` only.
- **A second failure domain appeared.** An in-process tool throwing was one call stack; a subprocess means a spawn failure, a crashed server, or a malformed protocol response are now separate things to reason about from "the tool logic itself was wrong." The `finally { await mcp.close(); }` around the whole loop exists specifically so a mid-run failure still tears the subprocess down instead of leaking it.

- **Real, measurable overhead.** Each `runTriageAgent()` call now pays subprocess startup plus a JSON-RPC round trip per tool call, instead of a function call. Fine for triaging one ticket; the honest answer to "how would this scale" is a connection pooled across tickets, not spawn-per-ticket — the kind of follow-up question this exact setup makes concrete instead of hypothetical.

13. Day 14 — Wrap-up

The 14-day plan is complete. Four working, independently-runnable versions, each tagged on the real commit where it shipped — `trriage-v1` through `trriage-v4` (see [README.md](./README.md) for the version table and how to run each one):

Tag	Commit	What it is
<code>trriage-v1</code>	<code>1137e78</code>	Baseline: one model call, retry+correction, scored eval harness
<code>trriage-v2</code>	<code>7f55380</code>	+ RAG: Voyage-embedded KB, cosine retrieval, <code>kb_citations</code>
<code>trriage-v3</code>	<code>0463fad</code>	+ Tool-use agent loop, capped iterations, full tool-call log
<code>trriage-v4</code>	<code>c5cc42e</code>	+ Real MCP server; agent discovers tools over the protocol

The throughline across all four stages, stated plainly: every version returns a typed outcome that never throws (`TriageOutcome`, then `AgentOutcome`), and every new capability was only trusted once it had been proven against something outside the code itself — gold labels for v1 (Days 1-6, not covered in this doc but the foundation everything else sits on), a hand-checked retrieval query for v2, a live forced `max_iterations` failure for v3, the MCP Inspector before the agent ever touched the server for v4. None of the four stages was accepted on "the types compile and it looks right" alone — each had a real run, with real output pasted back, before moving on. That discipline mattered more than any single technique (RAG, tool-use, MCP) — it's the actual answer to "how do you know your agent works," which is the harder and more interview-relevant question than "what's an agent."

14. Where to go next

- **Grow the dataset** to 40–60 tickets — the single highest-leverage move, since it stops the metrics from swinging on one ticket.
- **Day 14: wrap-up** — final README/LEARNINGS pass and git version tags (v1–v4) on the real commits that closed out each stage of the plan.
- **Pool the MCP connection** instead of spawning `mcp-server.ts` per ticket, if this ever needs to run `--all` across a whole dataset rather than one ticket at a time.
- **A real (or a second, non-Zendesk) MCP data source** — the current server still serves mock data; the protocol wiring is real, the backing data isn't yet.

- **Re-run `score.ts` with retrieval on** — Days 1–6's baseline (90% category / 90% severity) predates RAG; worth confirming retrieval didn't regress classification accuracy while improving `suggested_first_response`, not just assuming it didn't.
- **One surgical prompt fix** for the question-framed-bug confusion — but verify it does not ripple into other tickets.
- **A live-fired "not found" case** for `get_customer_account/check_recent_tickets` — craft a ticket with a plausible-looking but non-matching identifier (rather than none at all) to force the agent to actually call the tool and get a real miss, closing the coverage gap noted in Day 11.